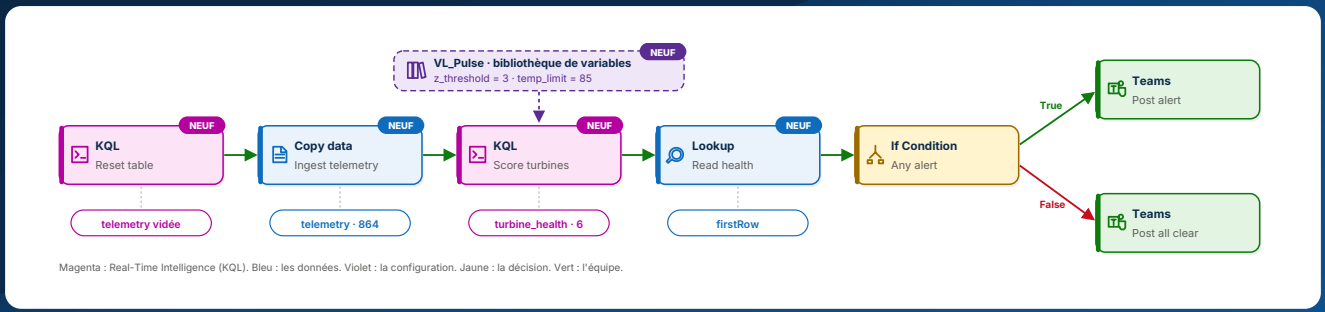
MICROSOFT FABRIC · DATA FACTORY · REAL-TIME INTELLIGENCE · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : ingérer, analyser en KQL, décider

VIDER · INGÉRER · ANALYSER · LIRE · DÉCIDER · PRÉVENIR

La télémétrie de six éoliennes entre dans un Eventhouse, une commande KQL juge chaque machine, le pipeline lit le verdict et prévient l'équipe dans Teams. Aucun seuil dans le pipeline : ils vivent dans une bibliothèque de variables.



7	4	864	6	0
activités	nouveautés	relevés	éoliennes	seuil dans le pipeline

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe

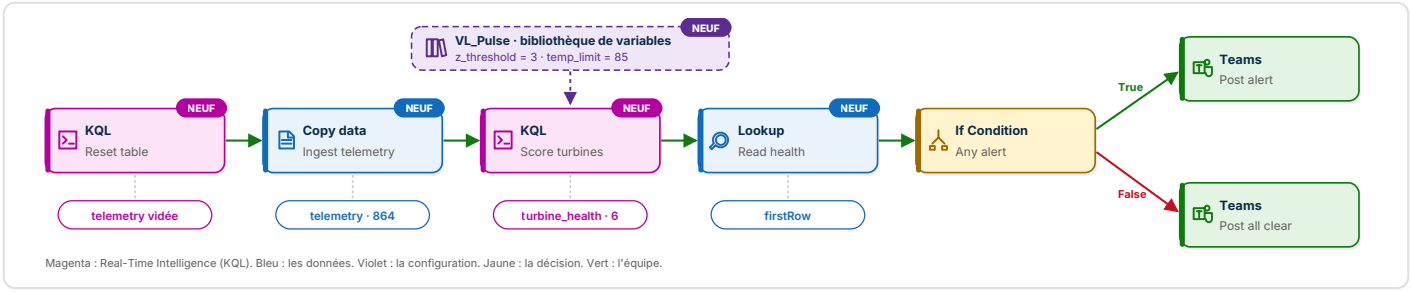
FICHIERS DU PROJET
Le fichier de télémétrie et expressions.txt, qui contient toutes les commandes KQL

DURÉE
Environ 40 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous construisez

Le parc éolien du Plateau lorrain compte six éoliennes. Chacune envoie un relevé toutes les dix minutes : vent, puissance, rotation, température du multiplicateur, vibration. Le pipeline vide la table du jour, ingère les 864 relevés dans un Eventhouse, demande à KQL de juger chaque machine, lit le verdict et poste dans Teams soit une alerte nominative, soit un « tout va bien ». Quatre nouveautés : **l'activité KQL**, **la copie vers une base KQL**, **le Lookup en KQL**, **la bibliothèque de variables**.



Les sept activités

ACTIVITÉ	TYPE	RÔLE
`Reset table`	KQL NEUF	Crée la table `telemetry` si elle manque, puis la vide : rejouer ne double jamais
`Ingest telemetry`	Copy data VERS KQL	Copie le CSV du lakehouse dans la base KQL de l'Eventhouse : 864 relevés
`Score turbines`	KQL NEUF	Une commande : énergie, vent moyen, température max, vibrations anormales et statut par éolienne, dans `turbine_health`
`Read health`	Lookup EN KQL	Une requête KQL résume le verdict en une ligne : nombre d'alertes, noms, production
`Any alert`	If Condition	Au moins une éolienne en alerte ?
`Post alert`	Teams, branche True	Nomme les éoliennes à inspecter et donne la production du jour
`Post all clear`	Teams, branche False	Six éoliennes en ordre de marche, production du jour

Le fichier

`telemetry_2026-10-12.csv` : le 12 octobre 2026, de 00:00 à 23:50, un relevé toutes les dix minutes pour `EOL-01` à `EOL-06`. Sept colonnes : `ts`, `turbine`, `wind_ms`, `power_kw`, `rotor_rpm`, `gearbox_temp_c`, `vibration_mms`.

CE QUE CACHE LA JOURNÉE	OÙ
Cinq pics de vibration, à 6 h 40 et 16 h 10	**EOL-04**
Multiplicateur qui chauffe jusqu'à 88,8 °C	**EOL-02**
Relevés au total	**864**

Ce qui est révolutionnaire

LE CALCUL VA À LA DONNÉE Le pipeline ne calcule rien. Il envoie une commande à l'Eventhouse, qui juge 864 relevés en une fraction de seconde, là où ils sont stockés.

ZÉRO SEUIL CODÉ EN DUR Le seuil de vibration et la limite de température vivent dans `VL_Pulse`. On les règle comme un paramètre d'application, le pipeline les lit à chaque exécution.

REJOUABLE Vider puis ingérer, remplacer puis lire : dix exécutions donnent dix fois 864 relevés.

Le pipeline en une image : la salle de contrôle d'un parc éolien

Le tableau blanc
Reset table efface le tableau de la veille avant la nouvelle journée.

Le relevé
Copy data recopie les 864 mesures des capteurs sur le tableau.

L'expert
KQL examine chaque machine d'un coup d'œil, selon les règles du jour.

Le règlement
VL_Pulse fixe les seuils, affiché au mur, modifiable par le chef.

Le chef d'équipe
If + Teams lit le verdict et prévient l'équipe d'intervention.

ÉTAPE 1

Créer l'Eventhouse, le lakehouse et la bibliothèque de variables

1 · OBJETS | 2 · VIDER ET INGÉRER | 3 · ANALYSER ET LIRE | 4 · DÉCIDER ET EXÉCUTER

1 EH_Pulse

EVENTHOUSE · NEUF

1. Espace de travail **Fabric_Pipeline_6**. **Nouvel élément**, **Eventhouse**, nom `EH_Pulse`.
2. Fabric crée en même temps une base KQL du même nom, `EH_Pulse`. C'est elle que le pipeline utilisera. Rien d'autre à faire : la table sera créée par le pipeline lui-même.

EVENTHOUSE, BASE KQL Le moteur de Real-Time Intelligence. Il est conçu pour la télémétrie : des millions de lignes horodatées, interrogées en KQL en quelques millisecondes.

2 LH_Pulse

LAKEHOUSE

1. **Nouvel élément**, **Lakehouse**, nom `LH_Pulse`. Dans **Fichiers**, **Nouveau sous-dossier** `inbox`.
2. Ouvrez `inbox`, **Charger**, **Charger des fichiers** : `telemetry_2026-10-12.csv`.

3 VL_Pulse

BIBLIOTHÈQUE DE VARIABLES · NEUF

1. **Nouvel élément**, **Bibliothèque de variables**, nom `VL_Pulse`.
2. **+ Nouvelle variable**, deux fois :

Nom	`z_threshold`
Type	`Nombre`
Valeur par défaut	`3`

Nom	`temp_limit`
Type	`Nombre`
Valeur par défaut	`85`

3. **Enregistrer**.

UNE RÈGLE MÉTIER, UN SEUL ENDROIT `z_threshold` : à partir de combien d'écarts-types une vibration est anormale. `temp_limit` : la température du multiplicateur à ne pas dépasser. Demain, un deuxième pipeline lira les mêmes valeurs.

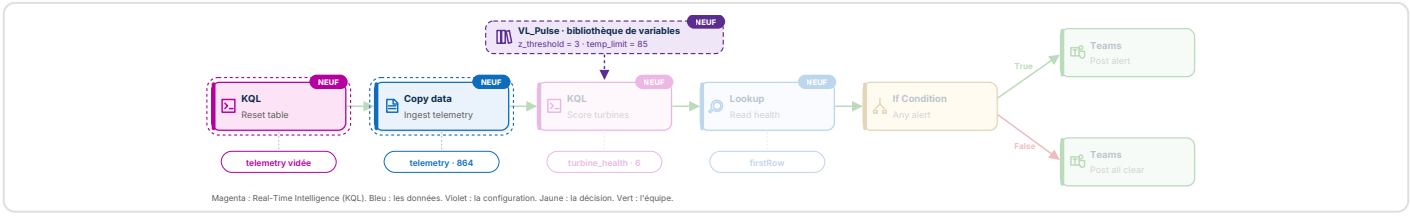
LE SCORE Z, EN UNE PHRASE Pour chaque éolienne, on mesure de combien d'écarts-types chaque vibration s'éloigne de sa moyenne. Au-delà de 3, c'est un pic que la machine n'a pas l'habitude de faire.

AVANT LE PIPELINE ☐ `EH_Pulse` et sa base KQL existent ☐ le CSV est dans `inbox` ☐ `VL_Pulse` : 3 et 85 ☐ tout est enregistré

ÉTAPE 2

Le pipeline : brancher la bibliothèque, vider, ingérer dans KQL

1 · OBJETS | 2 · VIDER ET INGÉRER | 3 · ANALYSER ET LIRE | 4 · DÉCIDER ET EXÉCUTER



4 PL_Pulse_Express et ses variables de bibliothèque

1. **Nouvel élément**, **Pipeline**, nom `PL_Pulse_Express`.
2. Cliquez sur le fond du canevas, onglet **Variables de bibliothèque**, **+ Nouveau**, deux fois :

Nom	`z_threshold`
Bibliothèque	`VL_Pulse`
Variable	`z_threshold`

Nom	`temp_limit`
Bibliothèque	`VL_Pulse`
Variable	`temp_limit`

COMMENT LE PIPELINE LES LIT Dans n'importe quelle expression : `@pipeline().libraryVariables.z_threshold`. La valeur est lue au lancement, jamais copiée dans le pipeline.

5 Reset table

KQL · NEUF

1. **Activités**, **KQL**, nom `Reset table`. **Paramètres**, **Connexion** : la base KQL `EH_Pulse`. Si une connexion est demandée : **Compte d'organisation**, **Se connecter**.
2. **Commande**, en texte simple :

```
.execute database script <|
.create-merge table telemetry (ts:datetime, turbine:string,
wind_ms:real, power_kw:long, rotor_rpm:real,
gearbox_temp_c:real, vibration_mms:real)
.clear table telemetry data
```

1. **Enregistrer**, **Exécuter**. La table `telemetry` apparaît, vide, dans `EH_Pulse`.

6 Ingest telemetry

COPY DATA · VERS KQL

1. **Copier les données**, **Ajouter au canevas**, nom `Ingest telemetry`, relié à la sortie verte de `Reset table`.
2. **Source** : `LH_Pulse`, racine **Fichiers**, chemin `inbox` / `telemetry_2026-10-12.csv`, format **DelimitedText**, **Première ligne comme en-tête** coché.
3. **Destination** : **Connexion** = la base KQL `EH_Pulse`, pas le lakehouse. **Table** : **Actualiser**, puis `telemetry`.
4. **Mappage**, **Importer les schémas** : les sept colonnes se font face par leur nom. L'avertissement orange sur `ts` annonce seulement la conversion du texte en date : ne changez rien.
5. **Enregistrer**, **Exécuter**. Sortie de `Ingest telemetry` : **864** lignes copiées.

LE TEXTE DEVIENT TYPÉ Le CSV n'a que du texte. C'est l'Eventhouse qui convertit à l'ingestion, d'après le schéma créé par `Reset table` : `datetime`, `long`, `real`. Aucun nettoyage côté pipeline.

VÉRIFIER EN DIX SECONDES Ouvrez `EH_Pulse`, la base KQL, **Interroger avec du code** :

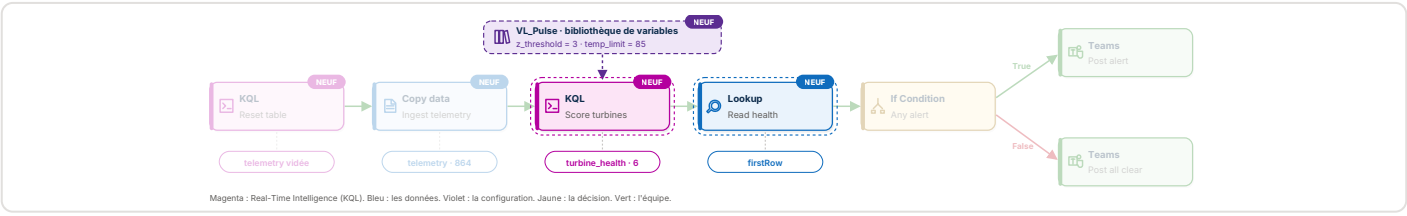
```
telemetry | count
```

Résultat : **864**. Exécutez le pipeline une seconde fois : toujours 864.

ÉTAPE 3

Juger chaque éolienne en KQL, lire le verdict

1 · OBJETS | 2 · VIDER ET INGÉRER | **3 · ANALYSER ET LIRE** | 4 · DÉCIDER ET EXÉCUTER



7 Score turbines

KQL · NEUF

1. **KQL**, nom `Score turbines`, relié derrière `Ingest telemetry`, même connexion `EH_Pulse`.
2. Cliquez dans **Commande**, puis **Ajouter du contenu dynamique**. Collez le bloc ci-dessous depuis `expressions.txt`, ligne **[Score turbines]**, et validez.

```
.set-or-replace turbine_health <|
let z = todouble('@{pipeline().libraryVariables.z_threshold}');
let tl = todouble('@{pipeline().libraryVariables.temp_limit}');
let s = telemetry | summarize mu = avg(vibration_mms), sd = stdev(vibration_mms) by turbine;
telemetry
| join kind=inner s on turbine
| summarize readings = count(), energy_mwh = round(sum(power_kw) / 6000.0, 2), avg_wind = round(avg(wind_ms), 1), max_temp =
max(gearbox_temp_c), vib_anomalies = countif(abs(vibration_mms - mu) / sd > z) by turbine
| extend status = iff(vib_anomalies >= 3 or max_temp > tl, "ALERTE", "OK")
```

Lignes 2 et 3
Le pipeline écrit les deux réglages de `VL_Pulse` dans la commande avant de l'envoyer : la commande reçoit `'3'` et `'85'`.

Lignes 4 à 6
Moyenne et écart-type de la vibration par éolienne, rattachés à chaque relevé : chaque machine est comparée à elle-même.

Lignes 7 et 8
Six lignes, une par éolienne. Statut `ALERTE` dès trois vibrations anormales ou une température au-delà de la limite.

8 Read health

LOOKUP · EN KQL

1. **Rechercher**, nom `Read health`, relié derrière `Score turbines`. **Paramètres**, **Connexion** : la base KQL `EH_Pulse`.
2. **Utiliser une requête** : **Requête**. Collez la ligne **[Read health]**. **Première ligne uniquement** coché.

```
turbine_health
| summarize alerts = countif(status == "ALERTE"), turbines =
count(), energy_mwh = round(sum(energy_mwh), 1), names =
strcat_array(array_sort_asc(make_list_if(turbine, status ==
"ALERTE")), ", ")
```

CE QUE LIT LE PIPELINE `output.firstRow.alerts` = 2, `.names` = `EOL-02, EOL-04`, `.turbines` = 6, `.energy_mwh` = 65.1.

AVANT DE DÉCIDER

- ☐ quatre activités reliées en vert
- ☐ la commande est en contenu dynamique
- ☐ Première ligne uniquement
- ☐ Enregistrer, le point a disparu

ÉTAPE 4

Décider, prévenir, puis changer la règle sans toucher au pipeline

1 · OBJETS | 2 · VIDER ET INGÉRER | 3 · ANALYSER ET LIRE | **4 · DÉCIDER ET EXÉCUTER**

9 Any alert — IF CONDITION

1. **Condition Si**, nom `Any alert`, derrière `Read health`. **Expression** :

```
@greater(activity('Read health').output.firstRow.alerts, 0)
```

10 Post alert, Post all clear — TEAMS

1. Case **True** : **Microsoft Teams**, nom `Post alert`. Case **False** : **Microsoft Teams**, nom `Post all clear`. Même équipe, canal **Général**, **Titre** : `Parc éolien du Plateau lorrain`.
2. **Message** : les lignes **[Post alert]** et **[Post all clear]** de `expressions.txt`. **Enregistrer**.

OBJECTIF
Deux exécutions, même fichier, même pipeline. Entre les deux, on ne change que deux valeurs dans `VL_Pulse`.

RÉSULTAT
864 relevés à chaque fois, 6 éoliennes jugées, une alerte nominative puis un « tout va bien ».

Exécution 1 — z_threshold = 3 · temp_limit = 85

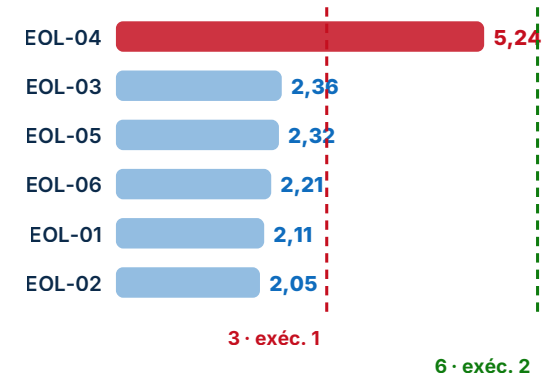
`Any alert` part en True. Dans Teams :

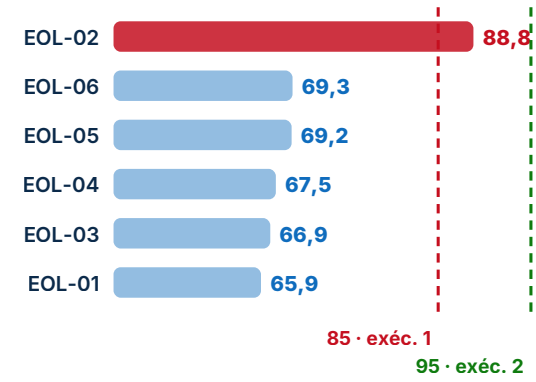
```
Alerte parc éolien : 2 éolienne(s) à inspecter :
EOL-02, EOL-04. Production du jour : 65.1 MWh.
```

Exécution 2 — z_threshold = 6 · temp_limit = 95

Dans `VL_Pulse`, remplacez 3 par `6` et 85 par `95`, **Enregistrer**. Relancez le pipeline sans l'ouvrir. Dans Teams :

```
Parc éolien : 6 éoliennes en ordre de marche.
Production du jour : 65.1 MWh.
```





LA PREUVE PAR DEUX EXÉCUTIONS Le pipeline n'a pas été ouvert entre les deux. Seule la bibliothèque a changé, et la décision a suivi. C'est ainsi qu'on sépare ce qui calcule de ce qui décide : la règle métier appartient au métier, le pipeline appartient à l'ingénieur.

Trois réflexes d'expert sur ce pipeline

Un jeu de valeurs par environnement

Dans `VL_Pulse`, **Jeux de valeurs** : « Strict » pour la production, « Tolérant » pour les essais. On active l'un ou l'autre, le pipeline ne change jamais.

Poser une règle Activator

Sur `turbine_health`, une règle Activator peut prévenir dès qu'un statut passe à `ALERTE`, sans attendre la prochaine exécution du pipeline.

Planifier toutes les dix minutes

Planifier le pipeline au rythme des relevés : le parc est jugé en continu, et la règle reste réglable à tout moment.

RÉFÉRENCE

Vérifier, retrouver une commande, débloquer

Vérifier dans l'Eventhouse

Ouvrez la base KQL `EH_Pulse`, **Interroger avec du code**, exécutez `turbine_health | order by turbine asc` après l'exécution 1 :

TURBINE	READINGS	ENERGY_MWH	AVG_WIND	MAX_TEMP	VIB_ANOMALIES	STATUS
EOL-01	144	8.89	7.1	65.9	0	OK
EOL-02	144	9.74	7.3	**88.8**	0	**ALERTE**
EOL-03	144	9.79	7.4	66.9	0	OK
EOL-04	144	11.41	7.6	67.5	**5**	**ALERTE**
EOL-05	144	12.57	7.8	69.2	0	OK
EOL-06	144	12.66	7.8	69.3	0	OK

864
relevés ingérés, à chaque exécution

5,24
écart maximal de vibration, EOL-04

88,8 °C
température maximale, EOL-02

65,1 MWh
produits par le parc dans la journée

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Connexion KQL : « accessToken » ou identifiants refusés	Type d'authentification **Compte d'organisation**, **Se connecter**. Jamais **Identité de l'espace de travail** ni **Principal du service**.
`Reset table` refuse `.execute database script`	Créez la table une fois dans **Interroger avec du code** avec la ligne `.create-merge table…`, puis gardez dans l'activité la seule commande `.clear table telemetry data`.
`Ingest telemetry` ne propose pas la table	La destination pointe vers le lakehouse. **Connexion** : la base KQL `EH_Pulse`. Si la table manque encore, exécutez `Reset table` seul, puis **Actualiser**.
`telemetry | count` donne 1 728	`Reset table` n'est pas relié devant le Copy, ou n'a pas tourné. Reliez-le en vert, relancez : 864.
`Score turbines` : « libraryVariables » introuvable	Les variables ne sont pas déclarées dans l'onglet **Variables de bibliothèque** du pipeline, ou le nom diffère. `z_threshold` et `temp_limit`, à la lettre.
KQL : « Syntax error » sur la ligne 2	La commande a été collée en texte simple : le `@{…}` n'est pas remplacé. Recollez-la dans **Ajouter du contenu dynamique**.
Bibliothèque de variables introuvable	Remplacez-la par deux paramètres de pipeline `z_threshold` et `temp_limit` (Float) et écrivez `pipeline().parameters` à la place de `pipeline().libraryVariables`.
Expression coupée après un copier-coller	Le PDF ajoute des espaces aux retours à la ligne. Copiez depuis `expressions.txt`.